



DISCLAIMER

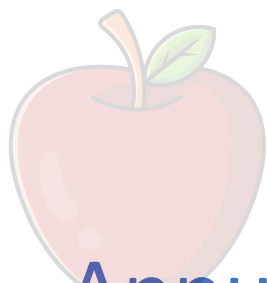
Il materiale contenuto nel drive è stato raccolto e richiesto tramite autorizzazione ai ragazzi frequentanti il corso di studi di Informatica dell'Università degli Studi di Salerno. Gli appunti e gli esercizi nascono da un uso e consumo degli autori che li hanno creati e risistemati per tanto non ci assumiamo la responsabilità di eventuali mancanze o difetti all'interno del materiale pubblicato.

Il materiale sarà modificato aggiungendo il logo dell'associazione, in tal caso questo possa recare problemi ad alcuni autori di materiale pubblicato, tale persona può contattarci in privato ed elimineremo o modificheremo il materiale in base alle sue preferenze.

Ringraziamo eventuali segnalazioni di errori così da poter modificare e fornire il miglior materiale possibile a supporto degli studenti.



CoScienze
Associazione



CoScienze

Associazione

Appunti ETC sul corso della Prof.ssa Luisa Gargano Parte 2

ESERCIZI SVOLTI, DIMOSTRAZIONI E SPIEGAZIONI

SCRITTO DA MATTIA LIMONE

.....	0
Dimostrare che un linguaggio è numerabile	2
L'insieme $\mathbb{R}$ non è numerabile	3
Esistono linguaggi che non sono Turing riconoscibili	4
A_{TM} è indecidibile	5
Esercizi linguaggi decidibili	8
Teorema di Rice.....	9
Riduzione $A_{TM} \leq_m L$	11
Riduzione $HALT_{TM} \leq_m L$	12
INDEPENDENT SET.....	13
VERTEX COVER	13
SET COVER.....	13
CIRCUIT SAT.....	13
3-SAT	13
HAM-CYCLE	14
DIR-HAM-CYCLE.....	14
k-COLOR	14
3-COLOR	14
k-REGISTER-ALLOCATION	14
SUBSET-SUM	14
Definizione classi di linguaggi.....	15
Dimostrazioni formali P -NP-coNP-EXP.....	17
Self-Reducibility.....	18
Riduzioni Problemi	22
Problemi di riflessione sulle riduzioni.....	26



CoScienze
Associazione

Dimostrare che un linguaggio è numerabile

$$X = \{(i,j) \mid i,j \in \mathbb{N}, i < j\}$$

$$f: X \rightarrow \mathbb{N};$$

	1	2	3	...	i	...	j
1	/	(1,2)	(1,3)	...	(1,i)	...	(1,j)
2	/	/	(2,3)	...	(2,i)	...	(2,j)
3	/	/	/	...	(3,i)	...	(3,j)
...	...	...	...	...	...	...	...
i	...	...	...	...	...	...	(i,j)
...	...	...	...	...	...	...	...
j	...	...	...	...	...	...	...

Troviamo dunque le seguenti coppie:

(1,2) = somma 3 -> 1 elemento

(1,3) = somma 4 -> 1 elemento

(1,4)(2,3) = somma 5 -> 2 elementi

(1,5)(2,4) = somma 6 -> 2 elementi

(1,6)(2,5)(3,4) = somma 7 -> 3 elementi

(1,7)(2,6)(3,5) = somma 8 -> 3 elementi

(1,j)(2,j)(3,j)(i,j) = somma i+j -> i+j elementi

Se i+j è dispari = $\left(\frac{i+j-1}{2} - 1\right)$ elementi più $\left(\frac{i+j-1}{2}\right)$

Aggiungo i che è il numero di elementi per arrivare a (i,j)

$$\text{Quindi } 2 \left(\frac{i+j-1}{2} - 1\right) \left(\frac{i+j-1}{2}\right) / 2 + i \Rightarrow \left(\frac{i+j-1}{2} - 1\right) \left(\frac{i+j-1}{2}\right) + i$$

Se i+j è pari = $\left(\frac{i+j-2}{2} - 1\right)$ elementi più $\left(\frac{i+j-1}{2}\right)$

Aggiungo i che è il numero di elementi per arrivare a (i,j)

$$\text{Allora } 2 \left(\frac{i+j-2}{2} - 1\right) \left(\frac{i+j+1}{2}\right) / 2 + \left(\frac{i+j-1}{2}\right) + i \Rightarrow \left(\frac{i+j}{2}\right) \left(\frac{i+j+1}{2}\right) + \left(\frac{i+j-1}{2}\right) + i$$

L'insieme $\mathbb{R}$ non è numerabile

Questo metodo si può facilmente applicare a qualsiasi altro insieme che si presuppone non numerabile. La risoluzione qui proposta è anche la risoluzione di un esercizio uscito ad una sessione. Il metodo dimostrativo è quello dell'argomento diagonale di Cantor.

Supponiamo che $(0, 1)$ dei numeri reali $\mathbb{R}$ sia numerabile. Quindi potremmo scrivere tutte le espansioni decimali dei reali in $(0, 1)$ in una lista:

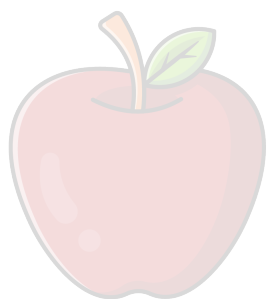
0. $a_{11} a_{12} a_{13} a_{14} a_{15} \dots$

0. $a_{21} a_{22} a_{23} a_{24} a_{25} \dots$

0. $a_{31} a_{32} a_{33} a_{34} a_{35} \dots$

0. $a_{41} a_{42} a_{43} a_{44} a_{45} \dots$

.....



	1	2	3	...	i	...
a_1	a_{11}	a_{12}	a_{13}	...	a_{1i}	...
a_2	a_{21}	a_{22}	a_{23}	...	a_{2i}	...
a_3	a_{31}	a_{32}	a_{33}	...	a_{3i}	...
...	...	...	...	...	...	...
a_i	a_{i1}	a_{i2}	a_{i3}	...	a_{ii}	...
...	...	...	...	...	...	...

Adesso definiamo un nuovo decimale $x = x_1 x_2 x_3 \dots x_i$ tale che

$$x_1 \neq a_{11}$$

$$x_2 \neq a_{22}$$

$$x_3 \neq a_{33}$$

$$x_i \neq a_{ii}$$

E così in avanti. Quindi l'espansione decimale di x differisce dall' i -esimo elemento della lista all' i -esima cifra decimale. Quindi rappresenta un elemento dell'intervallo $(0, 1)$ che non è nella lista e quindi non abbiamo una lista di numeri reali in $(0, 1)$.

Esistono linguaggi che non sono Turing riconoscibili

Prese $\{w_1, w_2, \dots\} \in \Sigma^*$ e $\{M_1, M_2, \dots\} = MdT$ su Σ che andiamo a numerare solo gli elementi $x_{i,j} = 1$ se $w_j \in L(M_i)$, 0 altrimenti, ottenendo perciò la seguente tabella.

	w_1	w_2	w_3	w_i	w_j
M_1	$x_{1,1}$	-	-	-	-
M_2	-	$x_{2,2}$	-	-	-
M_3	-	-	$x_{3,3}$	-	-
M_i	-	-	-	$x_{i,i}$	$x_{i,j}$
...					

Andiamo a definire un nuovo linguaggio, ovvero il linguaggio complemento degli elementi sulla diagonale.

$$L = \{w_i \in \Sigma^* \mid w_i \notin L(M_i)\}$$

La deduzione che ottengo dalla numerazione data in precedenza è che se l'elemento della diagonale è 1 $w_i \notin L(M_i)$, viceversa se è 0 $w_i \in L(M_i)$

Supponiamo ora un nuovo linguaggio $L = \{w_i \in \Sigma^* \mid w_i \notin L(M_i)\} = L(M_h)$

Se $w_h \in L$ allora $x_{h,h}$ vale 0 e di conseguenza $w_h \notin L(M_h) = L$

Se $w_h \notin L$ allora $x_{h,h}$ vale 1 e di conseguenza $w_h \in L(M_h) = L$

Otteniamo così una doppia contraddizione cercando di inserire una macchina il cui linguaggio è il complemento.

A_{TM} è indecidibile

Definiamo A_{TM} come $A_{TM} = \{\langle M, w \rangle \mid M \text{ è una TM e } M \text{ accetta } w\}$

La dimostrazione che A_{TM} è indecidibile porta come notevole conseguenza che i decisori sono un sottoinsieme dei riconoscitori.

Si parte da una macchina U

$U = \text{su input } \langle M, w \rangle, \text{dove } M \text{ è una TM e } w \text{ una stringa t.c}$

- simula M su input w
- se durante la computazione M va in stato di accettazione, accetta, se va in stato di rifiuto, rifiuta.

Banalmente questa macchina cicla sull'input $\langle M, w \rangle$ quando M cicla su w , motivo per il quale A_{TM} è indecidibile. Se ci fosse un modo per determinare che M non si ferma su w , sarebbe in tal caso in grado di rifiutare w .

Usiamo il metodo dimostrativo dell'argomento diagonale di Cantor come fatto per i numeri reali per dimostrare quanto detto.

Allora partendo da $A_{TM} = \{\langle M, w \rangle \mid M \text{ è una TM e } M \text{ accetta } w\}$

DIMOSTRAZIONE

Per assurdo assumiamo che A_{TM} sia decidibile e supponiamo H come decisore di A_{TM} . Di conseguenza sull'input $\langle M, w \rangle$ dove M è una TM e w una stringa, H si ferma ed accetta se M accetta w , e si ferma e rifiuta se M non accetta w .

$$H(\langle M, w \rangle) = \begin{cases} \text{accetta se } M \text{ accetta } w \\ \text{rifiuta se } M \text{ rifiuta } w \end{cases}$$

Ora costruiamo una nuova macchina di Turing D avente H come sottoprogramma. Questa nuova TM chiama H per determinare cosa fa M quando l'input di M è la sua stessa descrizione $\langle M \rangle$.

Una volta che D ha determinato questa informazione, essa fa il contrario. Cioè, rifiuta se M accetta ed accetta se M non accetta.

$D = \text{"Su input } \langle M \rangle, \text{dove } M \text{ è una TM:}$

1. Esegue H su input $\langle M, \langle M \rangle \rangle$.

2. Dà in output l'opposto di ciò che H dà in output. Cioè, se H accetta, rifiuta; e se H rifiuta, accetta."

$$D(\langle M \rangle) = \begin{cases} \text{accetta se } M \text{ non accetta } \langle M \rangle \\ \text{rifiuta se } M \text{ accetta } \langle M \rangle \end{cases}$$

Di conseguenza quando eseguiamo D con la sua stessa descrizione $\langle D \rangle$

$$D(\langle D \rangle) = \begin{cases} \text{accetta se } D \text{ non accetta } \langle D \rangle \\ \text{rifiuta se } D \text{ accetta } \langle D \rangle \end{cases}$$

Indipendentemente da ciò che D fa, essa è costretta a fare il contrario, il che è ovviamente una contraddizione. Quindi, nè la $TM D$ nè la $TM H$ possono esistere.

Costruiamo la diagonale per formalizzare la dimostrazione

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$
M_1	<u>A</u>	-	A	-
M_2	-	<u>R</u>	-	-
M_3	-	A	<u>A</u>	-
M_4	A	-	-	<u>R</u>

Eseguo H sugli input della tabella riportata sopra e ottengo, quindi dove la macchina M accettava H accetta, dove M rifiuta o va in loop (simbolo -) H rifiuta.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$
M_1	<u>A</u>	R	A	R
M_2	R	<u>R</u>	R	R
M_3	R	A	<u>A</u>	R
M_4	A	R	R	<u>R</u>

Infine, aggiungo D che per assunzione è una TM , non riporto le entrate al di fuori della diagonale perché poco interessanti e creerebbero solo confusione. Eseguendo H su questo nuovo input ottengo la tabella sottostante.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$
M_1	<u>A</u>	-	-	-	...	-
M_2	-	<u>R</u>	-	-	...	-
M_3	-	-	<u>A</u>	-	...	-
M_4	-	-	-	<u>R</u>	...	-
...	...	...	...	...	...	...
D	R	A	R	A	...	?

Subito si nota che la $TM D$ inverte la decisione della M_i in posizione i, i ovvero lungo la diagonale.

La contraddizione avviene in corrispondenza del punto interrogativo. Perché?

Perché l'entrata in corrispondenza del punto interrogativo dovrebbe essere il contrario di sé stessa come da costruzione qui riportata.

$$D(\langle D \rangle) = \begin{cases} \text{accetta se } D \text{ non accetta } \langle D \rangle \\ \text{rifiuta se } D \text{ accetta } \langle D \rangle \end{cases}$$



Esercizi linguaggi decidibili

Dati $k \geq 3$ linguaggi $L_1, \dots, L_k \subseteq \Sigma^*$ tale che

- 1) L_j è turing riconoscibile $\forall 1 \leq j \leq k$
- 2) I linguaggi sono a due a due disgiunti, $L_i \cap L_j = \emptyset, i \neq j$
- 3) Ogni stringa di Σ^* è in un linguaggio $L_j, \bigcup_{i=1}^n L_i = \Sigma^*$

Dimostrare che i singoli linguaggi sono decidibili, $\forall 1 \leq i \leq k$

DIMOSTRAZIONE

Sapendo che:

- 1) L_1 è turing riconoscibile, $\exists M_1$ tale che $L(M_1) = L_1$
- 2) I linguaggi sono a due a due disgiunti e costituiscono una partizione su Σ^*

$\forall w \in L_1, \exists$ una ed una sola MdT M_j tale che $w \in L(M_j) = L_j, 2 \leq j \leq k$

Decider multinastro M_1

- Simula sul nastro i la MdT M_i tale che $L_i = L(M_i)$
- Accetta $w \in \Sigma^*$ sse M_1 accetta w
- Rifiuta $w \in \Sigma^*$ sse M_j accetta $w, 2 \leq j \leq k$

Il procedimento viene eseguito su ogni altra macchina.

Dato che ogni decider multinastro M_i decide il corrispondente linguaggio M_i accettando se M_i accetta e rifiutando se $M_j, 1 \leq j \leq k, i \neq j$, accetta, tutti i linguaggi sono decidibili.

Teorema di Rice

Afferma che, per ogni proprietà non banale delle funzioni calcolabili, il problema di decidere quali funzioni soddisfino tale proprietà e quali no, è indecidibile.

Proprietà banale: proprietà che non effettua alcuna discriminazione tra le funzioni calcolabili, cioè che vale o per tutte o per nessuna.

Sia

$$P = \{\langle M \rangle \mid M \text{ è una MdT che verifica la proprietà } \rho\}$$

un linguaggio che soddisfa le seguenti due condizioni:

1) L'appartenenza di M a P dipende solo da $L(M)$, cioè

$$\forall M_1, M_2 \text{ MdT tali che } L(M_1) = L(M_2), \langle M_1 \rangle \in P \Leftrightarrow \langle M_2 \rangle \in P$$

2) P è una proprietà non banale, cioè

$$\exists M_1, M_2 \text{ MdT tali che } \langle M_1 \rangle \in P, \langle M_2 \rangle \notin P$$

Allora P è indecidibile.

1) $X = \{\langle M \rangle \mid M \text{ è MdT che RIFIUTA } ab \text{ e ACCETTA se termina con } a\}$

Consideriamo due MdT M_1 e M_2 t.c. che $L(M_1) = L(M_2)$ e mostriamo che

$$\langle M_1 \rangle \in L_P \text{ mentre } \langle M_2 \rangle \notin L_P$$

M_1 : Accetta le stringhe che terminano con a

Rifiuta la stringa ab

M_2 : Accetta le stringhe che terminano con a

Cicla sull'input ab

M_1 e M_2 riconoscono lo stesso linguaggio ma $\langle M_1 \rangle \in L_P$ e $\langle M_2 \rangle \notin L_P$, questo contraddice la prima proprietà del Teorema di Rice che quindi non può essere applicato

2) $FINITE-TM = \{\langle M \rangle \mid M \text{ è MdT che ACCETTA un linguaggio finito}\}$

- Proprietà non banale

$$L(M_1) = \{ab\}, \quad \langle M_1 \rangle \in FINITE - TM$$

$$L(M_2) = \{ab\}^*, \quad \langle M_2 \rangle \notin FINITE - TM$$

- Proprietà dipende dal linguaggio

$$HP: M_1, M_2 \text{ e } L(M_1) = L(M_2)$$

$$\langle M_1 \rangle \in FINITE - TM \Leftrightarrow |L(M_1)| < \infty \Leftrightarrow |L(M_2)| < \infty \Leftrightarrow \langle M_2 \rangle \in FINITE - TM$$

Sono verificate entrambe le proprietà dunque $FINITE-TM$ è indecidibile

3) $L = \{\langle M \rangle \mid M \text{ è MdT che ACCETTA le stringhe che iniziano per } ab\}$

- Proprietà non banale

$$L(M_1) = \Sigma^* / \{\varepsilon\}, \quad \langle M_1 \rangle \in L$$

$$L(M_2) = \emptyset, \quad \langle M_2 \rangle \notin L$$

- Proprietà dipende dal linguaggio

$$HP: M_1, M_2 \text{ e } L(M_1) = L(M_2)$$

$$\langle M_1 \rangle \in L \Leftrightarrow \forall abw, w \in \Sigma^*, abw \in L(M_1) \Leftrightarrow \forall abw, w \in \Sigma^*, abw \in L(M_2) \Leftrightarrow \langle M_2 \rangle \in L$$

Sono verificate entrambe le proprietà dunque L è indecidibile

4) $L_K = \{\langle M \rangle \mid M \text{ è MdT che ACCETTA esattamente } k \text{ stringhe}\}$

- Proprietà non banale

$$L(M_1) = \{w \mid w = a^i, 1 \leq i \leq k\}, \quad \langle M_1 \rangle \in L_K$$

$$L(M_2) = \{w \mid w = a^i, 0 \leq i \leq k\}, \quad \langle M_2 \rangle \notin L_K$$

- Proprietà dipende dal linguaggio

$$HP: M_1, M_2 \text{ e } L(M_1) = L(M_2)$$

$$\langle M_1 \rangle \in L_K \Leftrightarrow |L(M_1)| = k \Leftrightarrow |L(M_2)| = k \Leftrightarrow \langle M_2 \rangle \in L_K$$

Sono verificate entrambe le proprietà dunque L_K è indecidibile

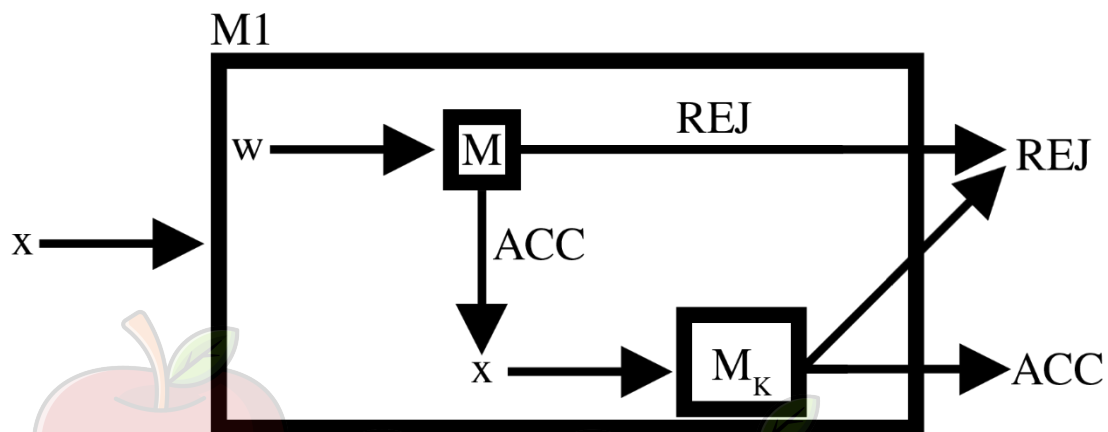
Riduzione $A_{TM} \leq_m L$

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ è una TM e } M \text{ accetta } w\}$$

$L = \text{Linguaggio con una determinata proprietà}$

Nell'esempio K-STRINGS-MDT

$$L(M_k) = \{1, \dots, k\}$$



M_1 su input x

- Esegue M su input w
- Se M rifiuta $\Rightarrow M_1$ rifiuta x
- Se M accetta $w \Rightarrow$ esegui M_k su input x
 - Se M_k rifiuta, M_1 rifiuta x
 - Se M_k accetta, M_1 accetta x

Di conseguenza:

$$\langle M, w \rangle \in A_{TM} \Rightarrow L(M_1) = L(M_k) \Rightarrow \langle M_1 \rangle \in L$$

$$\langle M, w \rangle \notin A_{TM} \Rightarrow L(M_1) = \emptyset \Rightarrow \langle M_1 \rangle \notin L$$

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow f(\langle M, w \rangle) = \langle M_1 \rangle \in L$$

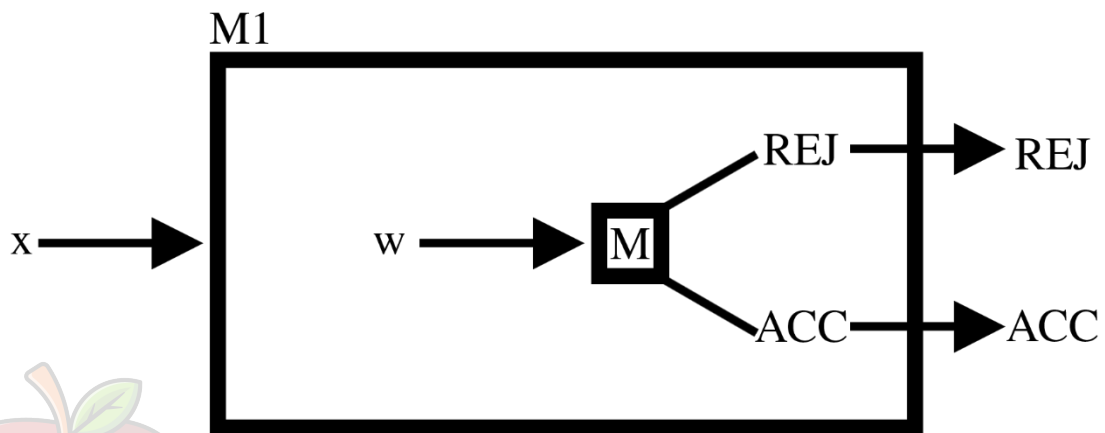
Riduzione $HALT_{TM} \leq_m L$

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ è una MdT e } M \text{ si arresta su } w\}$

$L = \text{Linguaggio con una determinata proprietà}$

Nell'esempio NO-LOOP

$L = \{\langle M \rangle \mid M \text{ è una MdT che si arresta su ogni input}\}$



M_1 su input x

- Esegue M su input w
- Se M rifiuta $\Rightarrow M_1$ rifiuta x
- Se M accetta $\Rightarrow M_1$ accetta x

$\langle M, w \rangle \in HALT_{TM} \Rightarrow M \text{ si arresta su } w$

$\Rightarrow M \text{ si arresta } \forall x$

$\Rightarrow \langle M_1 \rangle \in L$

$\langle M, w \rangle \notin HALT_{TM} \Rightarrow M \text{ va in loop su } w$

$\Rightarrow M \text{ va in loop } \forall x$

$\Rightarrow \langle M_1 \rangle \notin L$

$\langle M, w \rangle \in HALT_{TM} \Leftrightarrow f(\langle M, w \rangle) = \langle M_1 \rangle \in L$

INDEPENDENT SET

Dato un grafo $G = (V, E)$, e un intero k , esiste un sottinsieme di vertici $S \subseteq V$ tale che $|S| \geq k$, e per ogni edge al più uno di suoi estremi è in S ?

$$\forall (u, v) \in E$$

$$\text{se } u \in IS \Rightarrow v \notin IS$$

$$\text{se } v \in IS \Rightarrow u \notin IS$$

VERTEX COVER

Dato un grafo $G = (V, E)$, e un intero k , esiste un sottinsieme di vertici $S \subseteq V$ tale che $|S| \leq k$, e per ogni edge almeno uno dei suoi estremi è in S ?

$$\forall (u, v) \in E$$

$$\text{o } u \in VC$$

$$\text{o } v \in VC$$



Coscienze

SET COVER

Dato un insieme U di elementi, una collezione $S_1, S_2, \dots, S_m$ di sottoinsiemi di U e un intero k , esiste insieme di collezioni di cardinalità $\leq k$ la cui unione è uguale a U ?

$$\forall S_i \subseteq U, 1 \leq i \leq k$$

$$\sum_{i=1}^k S_i = U$$

CIRCUIT SAT

Dato un circuito combinatoriale composto da porte AND, OR, e NOT, c'è un modo di settare gli input del circuito in modo che l'output sia "TRUE" o "1"?

3-SAT

Una 3-CNF o 3-SAT è una formula Booleana costituita da clausole collegate dall'operatore AND e ogni letterale al suo interno collegato dall'operatore OR, ogni clausola contiene esattamente 3 letterali.

Esempio: $(x_1 + x_2 + x_3) \cdot (\bar{x}_1 + x_4 + x_6) \cdot (x_2 + \bar{x}_5 + \bar{x}_3) \cdot (x_1 + \bar{x}_3 + \bar{x}_6)$.

Un assegnamento di variabili booleana in una formula è detto assegnamento di verità

Un assegnamento è detto soddisfacente, se fa in modo tale che la formula sia valutata come "TRUE".

Una 3-SAT è detta soddisfacibile se ha un assegnamento soddisfacente.

Data una formula Booleana in CNF tale che ogni clausola ha 3 letterali, è la formula soddisfacibile?

HAM-CYCLE

Dato un grafo non orientato $G = (V, E)$, esiste un ciclo semplice Γ che contiene ogni nodo in V ?

DIR-HAM-CYCLE

Dato digrafo $G = (V, E)$, esiste una ciclo orientato semplice Γ che contiene ogni nodo in V ?

k-COLOR

Dato un grafo non orientato G esiste un modo di colorare I nodi usando k colori in modo che nodi adiacenti NON hanno lo stesso colore?

3-COLOR

Dato un grafo non orientato G esiste un modo di colorare i nodi in rosso, blu, o verde in modo che nodi adiacenti non hanno lo stesso colore?

k-REGISTER-ALLOCATION

Date le variabili di un programma assegna le variabili ai registri della macchina in modo tale che non più di k registri sono utilizzati e non esistono 2 variabili che sono utilizzate contemporaneamente e sono assegnate allo stesso registro

SUBSET-SUM

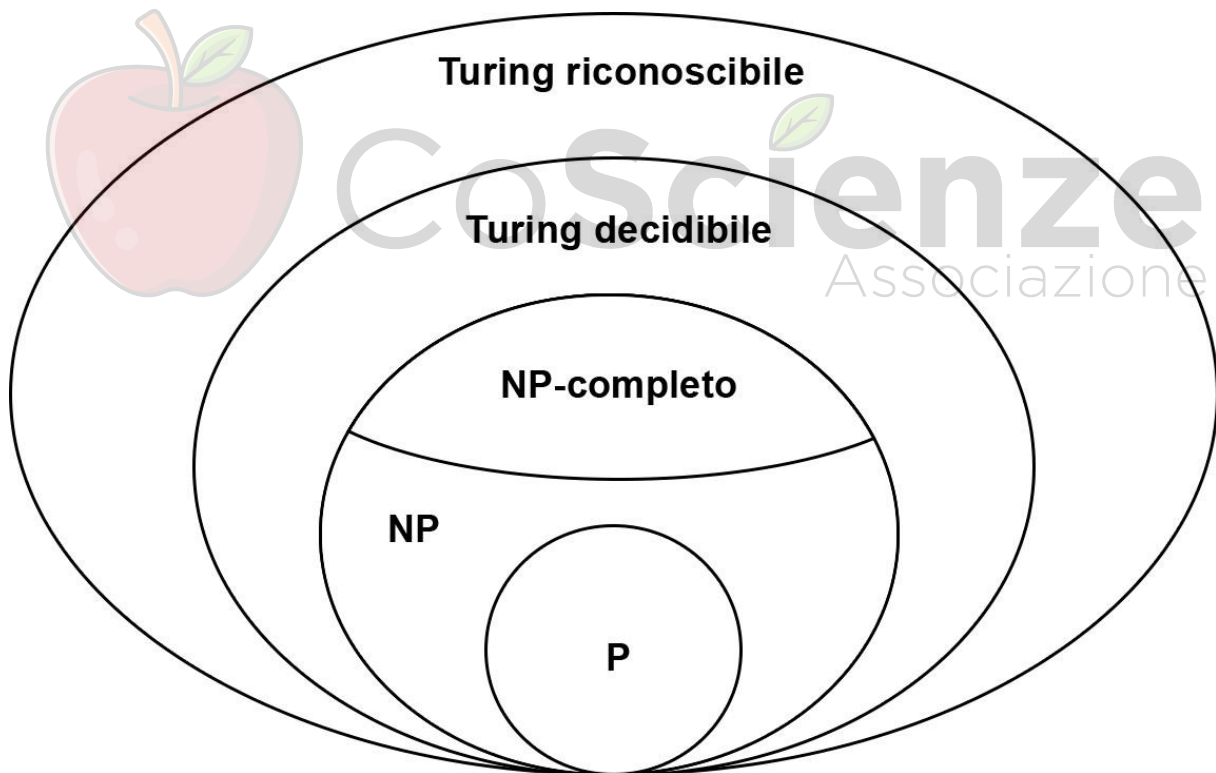
Dati numeri naturali $x_1, \dots, x_n$ e intero W , esiste sottoinsieme la cui somma è esattamente W ?

Definizione classi di linguaggi

Disegnare un diagramma che mostra le relazioni tra le seguenti classi di linguaggi (si ricordi che il linguaggio corrispondente ad un problema di decisione X è $L_X = \{\langle x \rangle \mid x \text{ è un'istanza SI per } X\}$):

- Linguaggi Turing riconoscibili,
- Linguaggi Decidibili,
- Classe dei linguaggi corrispondenti a problemi in P ,
- Classe dei linguaggi corrispondenti a problemi in NP ,
- Classe dei linguaggi corrispondenti a problemi NP -completi

Sotto l'assunzione che $P \neq NP$; Motivare brevemente le risposte (un diagramma non giustificato non viene valutato).



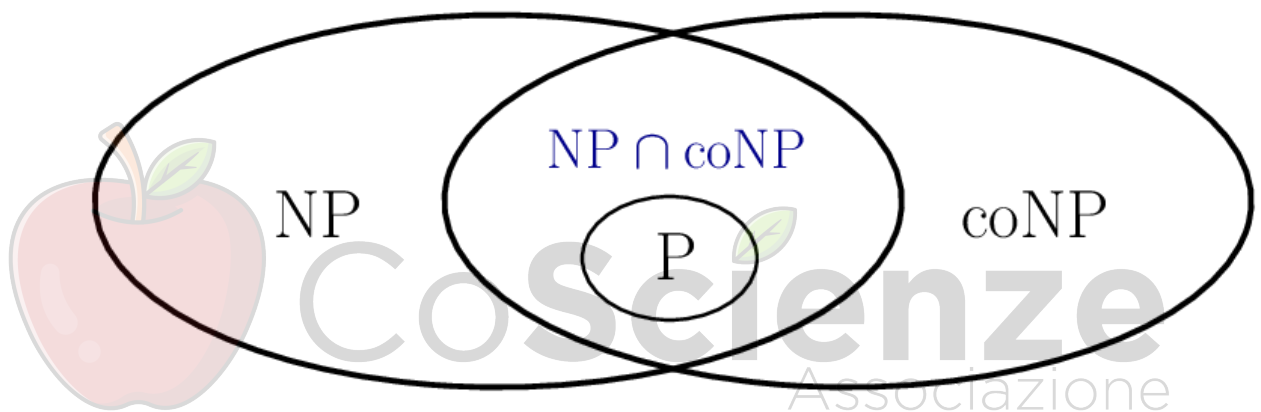
- 1) I linguaggi Turing decidibili sono inclusi nei linguaggi Turing riconoscibili perché come dimostrato da A_{TM} esiste un'intera classe di linguaggi che è riconoscibile ma non decidibile. Un linguaggio è decidibile sse è sia Turing riconoscibile che co-Turing riconoscibile.

- 2) Un linguaggio è in NP sse esso viene deciso in tempo polinomiale da una macchina di Turing non deterministica per cui la classe dei linguaggi corrispondenti a NP è inclusa nei Turing decidibili.
- 3) P è la classe dei linguaggi per cui l'appartenenza è decidibile in tempo polinomiale. NP è la classe dei linguaggi per cui l'appartenenza è verificabile in tempo polinomiale.

Ogni problema P dato che può essere deciso in tempo polinomiale è altrettanto verificabile in tempo polinomiale, di conseguenza $P \subseteq NP$.

- 4) Infine, la classe NP-completo è inclusa in NP in quanto identifica quei problemi che appartengono a NP e che dato un problema $X \in NP$ per ogni problema Z in NP vale la riduzione $Z \leq_p X$.

Nel caso invece venisse chiesta la relazione NP – coNP – P



1. P è la classe dei linguaggi per cui l'appartenenza è decidibile in tempo polinomiale. NP è la classe dei linguaggi per cui l'appartenenza è verificabile in tempo polinomiale.
Ogni problema P dato che può essere deciso in tempo polinomiale è altrettanto verificabile in tempo polinomiale, di conseguenza $P \subseteq NP$.
2. La classe P è chiusa rispetto al complemento. Se un linguaggio L è decidibile in tempo polinomiale da una macchina M1 basta considerare la macchina M2 che inverte la condizione di Accept e Reject.
Di conseguenza $P \subseteq coNP$.
3. coNP è la classe dei problemi decisionali P per cui $P \in NP$, P come visto è chiusa per il complemento. Se $P = NP$, allora NP è chiusa per il complemento cioè $NP = co-NP$, ma essendo $P \neq NP$ allora $NP \neq co-NP$.

Dimostrazioni formali P-NP-coNP-EXP

a) $P \subseteq NP$

$L \in P$, per definizione $\exists$ algoritmo M che decide L in tempo polinomiale. Consideriamo l'algoritmo di verifica V che su input y

1. $y \neq \langle w, \varepsilon \rangle$, rifiuta y
2. $y = \langle w, \varepsilon \rangle$ con w stringa, ε certificato simula M su $\langle w \rangle$ e accetta y sse M accetta $\langle w \rangle$, rifiuta y sse M rifiuta $\langle w \rangle$.

V verifica L in tempo polinomiale

b) $NP \subseteq EXP$

$L \in NP$, per definizione $\exists$ certificatore polinomiale $C(s,t)$ per L . Per risolvere L su input s , definiamo un algoritmo per decidere L

1. Genera tutte le possibili stringhe t con $|t| \leq p(|s|)$
2. Se il certificatore restituisce SI per qualche certificato t , allora l'algoritmo restituisce SI

Il certificatore ha tempo polinomiale, ma l'algoritmo è esponenziale.

c) $P \subseteq coNP$

$$coNP = \{L | \bar{L} \in NP\}$$

$$L \in P \Rightarrow^1 \bar{L} \in P \Rightarrow^2 \bar{L} \in NP \Rightarrow^3 L \in coNP \Rightarrow P \subseteq coNP$$

- 1) P è chiuso rispetto al complemento
- 2) $P \subseteq NP$
- 3) Per definizione di $coNP$

Self-Reducibility

Problema di ricerca $\leq_p$ Problema di decisione

Molti problemi di ricerca sono solo polinomialmente più difficili del corrispondente problema decisionale, ovvero: qualsiasi soluzione efficiente del problema decisionale può essere utilizzata per risolvere il problema di ricerca in modo efficiente. Questa si chiama self-reducibility.

RicSat $\leq_p$ Sat

Se $\emptyset$ è soddisfacibile, determinare l'assegnazione di verità alle variabili di $\emptyset$ che la rende vera, altrimenti segnalare che $\emptyset$ non è soddisfacibile.

Per ricercare i valori delle variabili di SAT il procedimento è semplice. Creo una copia di $\emptyset$ che chiamo ψ , ciclando da 1 fino a n che è il numero di variabili presente richiedo a SAT se il problema è soddisfacibile con la variabile x_i impostata una volta a false e una volta a true. Se mi risponde SI alla prima allora imposto a false, true altrimenti. Al secondo ciclo passerò x_{i-1} precedentemente calcolato e la variabile x_i impostata una volta a false e una volta a true, e così via. Una volta terminato il ciclo faccio un test sulla formula $\emptyset$ con i valori trovati e se restituisce SI riporto l'insieme dei valori

Mostrare RicSat $\leq_p$ Sat

1	$\psi(x_1, \dots, x_n) \leftarrow \emptyset(x_1, \dots, x_n)$
2	for $i \leftarrow 1$ to n do
3	$\psi' \leftarrow \psi(x_1=0)$
4	$\psi'' \leftarrow \psi(x_1=1)$
5	if (SAT(ψ')) then
6	$a_i = 0$
7	$\psi = \psi(0, x_2, \dots, x_n)$
8	elseif (SAT(ψ'')) then
9	$a_i = 1$
10	$\psi = \psi(1, x_2, \dots, x_n)$
11	else
12	return "IMPOSSIBILE"
13	if ($\emptyset(a_1, \dots, a_n) = 1$) then
14	return ($a_1, \dots, a_n$)
15	else
16	return "IMPOSSIBILE"

Longest Path $\leq_p$ Path

Path problema di decisione se esiste un cammino k in un grafo.

Longest Path determinare il cammino massimo in un grafo.

Calcolo il K massimo con n chiamate al problema di decisione aumentando di 1 la variabile i, la chiamata restituisce NO quando non esiste un cammino di quella grandezza, ciò significa che i - 1 era un valore accettabile

Chiamo il problema di decisione eliminando un edge alla volta per vedere se esso è rilevante nel cammino, se mi risponde SI quell'edge non è rilevante e allora salvo il resto in Edges

In Edges ho man mano salvato i vari edge che erano necessari al path di grandezza massima k calcolato precedentemente

1	<i>Vertex</i> <- V
2	<i>Edges</i> <- E
3	<i>n</i> <- E
4	for <i>i</i> <- 0 to <i>n</i> do {
5	if (PATH(<i>Vertex</i> , <i>Edges</i> , <i>i</i>))
6	//non fare nulla vai al prossimo <i>i</i> ++
7	else
8	<i>i</i> <- <i>n</i> + 1 // condizione di uscita dal ciclo successivo
9	<i>k</i> <- <i>i</i> - 1 // ecco il massimo numero di edge
10	}
11	if (<i>k</i> = 1)
12	return "OGNI COPPIA DI NODI È UN LONGEST PATH"
13	if (<i>k</i> = 0)
14	return "NON ESISTE UN PATH"
15	for <i>i</i> <- 1 to <i>n</i> do {
16	if (PATH(<i>Vertex</i> , <i>Edges</i> - <i>e</i> (<i>i</i>), <i>k</i>)) {
17	<i>Edges</i> <- <i>Edges</i> - <i>e</i> (<i>i</i>)
18	else
19	//non fare nulla vai al prossimo edge
20	}
21	return <i>Edges</i>

Max Independent Set $\leq_p$ Independent Set

Independent Set problema di decisione, dato un grafo $G = (V,E)$ ed un intero k esiste un IS di dimensione almeno k ?

Max Independent Set problema di ricerca, dato un grafo $G = (V,E)$ determinare il massimo IS.

Calcolo il K massimo con n chiamate al problema di decisione aumentando di 1 la variabile i , la chiamata restituisce NO quando non esiste un cammino di quella grandezza, ciò significa che $i - 1$ era un valore accettabile

Chiamo il problema di decisione eliminando uno per volta un nodo e i suoi vicini per vedere se esso è rilevante nell'IS con $k-1$, se mi risponde SI quell'edge è rilevante e allora lo salvo in I . Alla fine ritorno I .

1	$max \leftarrow - V $
2	$vertex \leftarrow V$
3	$edges \leftarrow E$
4	for $i \leftarrow 0$ to max do {
5	if $(IS(G,i))$
6	//non fare nulla vai al prossimo $i++$
7	else
8	$k \leftarrow i - 1$ // ecco il massimo numero di edge
9	}
10	if $(k = 1)$
11	return "OGNI VERTICE DA SOLO È IS"
12	if $(k = 0)$
13	return "NON ESISTE UN IS"
14	for $i \leftarrow 1$ to max do {
15	if $(IS(G/v_i \text{ e i suoi vicini}, k-1))$
16	$I \leftarrow I \cup v_i$
17	$G \leftarrow G/v_i \text{ e i suoi vicini}$
18	}
19	return I
20	
21	

Search Ham Cycle $\leq_p$ Ham Cycle

HamCycle, dato un grafo G esiste un ciclo semplice?

SearchHamCycle, dato un grafo G qual è un ciclo semplice?

Chiedo se esiste un Ham Cycle se risponde SI proseguo altrimenti termino

Rimuovo un edge alla volta. Se esiste il ciclo Hamiltoniano senza il dato edge, significa che esso non è rilevante nel ciclo, allora posso eliminarlo dall'insieme degli Edge.

Altrimenti esso fa parte degli edge che costituiscono il ciclo e non lo rimuovo.

A termine computazione in Edge saranno rimasti solo gli Edge rilevanti al ciclo.

1	<i>vertex</i> $\leftarrow V$
2	<i>edges</i> $\leftarrow E$
3	<i>n</i> $\leftarrow E $
4	<i>if</i> (!HAMCYCLE(<i>Vertex</i> , <i>Edges</i>))
5	<i>return</i> "NON ESISTE UN CICLO HAMILTONIANO"
6	<i>for</i> <i>i</i> $\leftarrow 1$ <i>to</i> <i>n</i> <i>do</i> {
7	<i>if</i> (HAMCYCLE(<i>Vertex</i> , <i>Edges</i> - <i>e_i</i>))
8	<i>Edges</i> $\leftarrow$ <i>Edges</i> - <i>e_i</i>
9	<i>else</i>
10	// non fare nulla vai al prossimo edge
11	}
12	<i>return</i> <i>Edges</i>

Riduzioni Problemi

SET-COVER $\leq_p$ PALESTRA

Una palestra cerca istruttori in grado di coprire corsi nelle discipline sportive $D1, \dots, Dm$. Gli istruttori candidati sono $I1, \dots, Im$ dove l'istruttore Ii è in grado di insegnare un insieme di discipline $S_i \subseteq \{D1, \dots, Dn\}$ (per ogni $i = 1, \dots, m$). Se il direttore della palestra intende arruolare al massimo k istruttori, risulta possibile ricoprire tutte le discipline $D1, \dots, Dn$? Chiamare il problema descritto PALESTRA; formalizzarlo (indicare input e output desiderato) e mostrare che esso risulta NP-completo.

[Aiuto. Si può sfruttare il fatto che il problema Set-Cover risulta NP-completo.]

Dimostrazione

Palestra $\in$ NP

Certificato = k istruttori $I_1, \dots, I_k$

Certificatore = Controlla $S_{i1} \cup \dots \cup S_{ik}$

Istanza di SetCover

$U: \{u_1, \dots, u_n\}$

$S_1, \dots, S_n \subseteq U$

k

Istanza di Palestra

$D = U$

$I_1, \dots, I_n \subseteq D$ istruttori che insegnano S_i discipline

k massimi istruttori

$S_{i1}, \dots, S_{ik}$ è una soluzione di Set-Cover sse $S_{i1} \cup \dots \cup S_{ik} = U = D$, il che equivale a dire (per come abbiamo costruito l'istanza di PALESTRA) che gli istruttori $I_{i1}, \dots, I_{ik}$ coprono tutte le discipline.

Quindi $S_{i1}, \dots, S_{ik}$ è una soluzione di Set-Cover sse $I_{i1}, \dots, I_{ik}$ è una soluzione di PALESTRA.

INDEPENDENT SET $\leq_p$ FESTA-PER-X

FESTA-per-X: Una persona riceve la visita di un vecchio amico X, per l'occasione vuole organizzare una festa scegliendo tra tutti suoi amici almeno k persone che si conoscono tra di loro e che conoscono X. Mostrare che FESTA-per-X è NP-completo.

[Sugg. Usare una riduzione da Clique o INDEPENDENT-SET]

DIMOSTRAZIONE

Certificato: insieme S e k

Certificatore: controlla $|S|=k$ e che $\forall u, v \in S, (u,v) \in E$

Istanza di Independent Set

S un Independent Set su $G = (V,E)$, $|S| = k$, $x \notin V$

Istanza di FESTA -per-X

Costruiamo il complemento di G chiamato G^c , dove $(G^c = (V \cup \{x\}, E^c))$ e $\forall u \in V, (u, x) \in E^c$

S è un IS di taglia k sse FESTA-per-X è S.

$S \in IS, |S| = k \Leftrightarrow S \in FESTA\text{-per-X}, |S| = k$

S è una clique in G^c per costruzione, quindi in S ci sono k persone che si conoscono e conoscono x. Quindi possiamo invitare chi è in S.

$\forall u, v \in S, \exists (u,v) \in E^c \Rightarrow \nexists (u,v) \in E \Rightarrow S \in IS \text{ in } G$

CLIQUE $\leq_p$ SUBIS

Due grafi $G=(V,E)$ e $G'=(V',E')$ sono isomorfi se $|V| = |V'|$ ed esiste una biezione $f: V \rightarrow V'$ tale che $(u,v) \in E$ sse $(f(u),f(v)) \in E'$

Dati due grafi $G=(V,E)$, $H=(U,F)$, G contiene un sottografo isomorfo ad H ?

DIMOSTRAZIONE

Certificato: $H(V',E')$

Certificatore: $\forall (u, v) \in E$ verifica che $(f(u),f(v)) \in E'$

Istanza di Clique

$H_G = (U_G, F_G)$ su k vertici; $|U_G| = k$; $\forall u, v \in U_G, (u,v) \in F_G$

Istanza di Subis

Grafo G e un suo sottografo G' isomorfo a H_G

G' è una clique per costruzione di H_G :

1) H_G ha k vertici per costruzione

2) $\forall u, v \in U_G, (u,v) \in F_G$

Viceversa

Consideriamo una k clique in G . Ogni arco (u,v) incluso nella clique $\in E$

Consideriamo il sottografo $C=(V',E')$ indotto dalla clique. Diciamo che C è isomorfo ad H_G in quanto $\forall u, v \in U_G, (u,v) \in E'$

Quindi $\langle G, H_G \rangle \in \text{Subis}$

INDEPENDENT SET $\leq_p$ DIF-SET

Una ditta mantiene una matrice A dove le righe corrispondono ai suoi clienti e le colonne ai prodotti che vende: $A[i, j]$ specifica la quantità di prodotto j acquistata da i

Due clienti sono diversi se per ogni prodotto al massimo uno dei due lo ha comprato.

Definiamo il problema DIF-SET:

Data una matrice $n \times m$ e un intero $k \leq n$, esiste un sottoinsieme di almeno k clienti a due a due diversi?

Dimostrare che DIF-SET è NP-completo.

[Sugg. Usare una riduzione da INDEPENDENT-SET]

DIMOSTRAZIONE

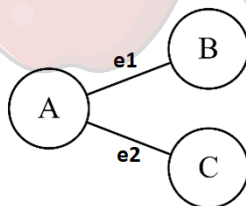
Certificato: matrice A , k e insieme $S = (n_i, \dots, n_k)$

Certificatore: controlla che $|S| \leq k$, e che $\forall [x, j] \neq 0 \Rightarrow [y, j] = 0, 1 \leq y \leq k, y \neq x$

Istanza di IS

S un Independent Set su $G = (V, E)$, $|S| = k$

$S = \{B, C\}$, $k = 2$



Istanza di DIF-SET

V = clienti, E = stesso prodotto acquistato da due clienti e k

$\forall (i, j) \in E, [i, m] = 1 \text{ e } [j, m] = 1 \text{ in } A, 0 \text{ altrimenti}$

La matrice derivante ad esempio dal grafo sovrastante è la seguente

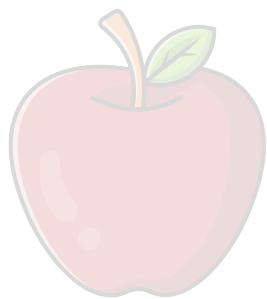
	e1	e2
A	1	1
B	1	0
C	0	1

B e C non avendo prodotti acquistati in comune sono clienti diversi

S è un IS di dimensione $k \Leftrightarrow S$ è un insieme di clienti diversi di dimensione k
nella matrice A per costruzione

Viceversa

S è un DIF-SET di dimensione $k \Leftrightarrow S$ è un IS di dimensione k dato che non
esiste l'arco che li collega per
costruzione della matrice



CoScienze
Associazione

3-SAT $\leq_p$ DIFF-SAT

Considerare il problema *DIFF – SAT* che prende in input un'istanza Φ di *SAT* e restituisce vero se e solo formula Φ ammette un'assegnazione di verità alle variabili avente l'ulteriore proprietà che ogni clausola formata da almeno due letterali contiene sia un letterale vero che un letterale falso.

Mostrare che *DIFF – SAT* è NP-Completo.

[Sugg. Usare una riduzione da 3-SAT]

DIMOSTRAZIONE

Certificato: assegnamento di verità

Certificatore: Controlla che l'assegnamento di verità sia soddisfatto e che almeno un letterale per clausola sia false e uno sia true.
 $O(|\phi|)$

Istanza di 3-SAT

ϕ in CNF

Istanza di DIFF-SAT

$\forall C_i$ aggiungiamo una variabile y_i alla clausola C_i e una nuova clausola contenente $\bar{y}_i$

Se ϕ è soddisfacibile usiamo lo stesso assegnamento di verità su ϕ' ponendo ogni $y_i = 0$, così facendo le clausole derivate dalle originali saranno soddisfatte e la condizione di DIFF-SAT di avere almeno un letterale true e false per clausole con almeno due letterali è altrettanto soddisfatto.

Se ϕ non è soddisfacibile allora ϕ' non è soddisfacibile in quanto la clausola derivata dalla originale con l'aggiunta di y_i a patto di porre y_i a 1 ma ciò viola le clausole $\bar{y}_i$.

Problemi di riflessione sulle riduzioni

Dato $X \leq_p Y$

1) Se Y è NP-completo lo è anche X

Non si sa, affinché sia vera:

$$X \in \text{NP}, \forall Z \in \text{NP } Z \leq_p X$$

2) Se Y è NP-Completo e $X \in \text{NP}$, allora X è NP-Completo

Non si sa, affinché sia vera:

$$\forall Z \in \text{NP } Z \leq_p X$$

3) Se X è NP-Completo e $Y \in \text{NP}$, allora Y è NP-Completo

Vero, per definizione di NP-Completezza, ovvero:

$$X \in \text{NP}, \forall Z \in \text{NP } Z \leq_p X$$

4) X e Y non possono essere entrambi NP-Completi

Falsa, banalmente se $X \in \text{NP-Completo}$ e $Y \in \text{NP}$ sono verificate le condizioni di NP-Completezza e quindi $Y \in \text{NP-Completo}$.

5) Se X è in P , allora Y è in P

Non si sa, possiamo solo dire che Y è almeno in P , ma ciò non identifica con precisione il massimo insieme di cui può far parte, ma solo il limite inferiore.

6) Se Y è in P , allora X è in P

Vero, convertendo tramite trasformazione polinomiale un'istanza di X in istanza di Y e risolvendo l'istanza di Y tramite il suo algoritmo polinomiale risolvo X in tempo polinomiale.